

CMU INSTRUCTIONAL MODULE

GitHub Fundamentals for Beginners

A CMU Format Instructional Module



by **RINANTE M. BUNTOD**

Semester Edition for **BSIT students with no prior Git experience**

Preface

GitHub Fundamentals for Beginners is a semester learning module for BSIT students who are new to version control, command-line Git, and collaborative development on GitHub. It begins with a concrete model of how files move from a working folder into a recorded history, then develops the skills needed to collaborate, automate checks, publish a static site, and recover from common mistakes.

The module uses one continuing student-portfolio project so that every activity contributes to a visible body of work. Students should complete the diagnostic questions before reading each unit, follow the guided activity in a practice repository, and submit only the evidence named in the performance task. Private keys, access tokens, passwords, and other secrets must never be submitted.

Table of Contents

Preface.....	2
Table of Contents	2
List of Figures.....	8
Glossary	9
Semester Roadmap.....	11
Unit 1: Introduction to Git and GitHub	13
Overview.....	13
Introduction and Rationale	13
Learning Objectives	13
Learner Instructions	13
Materials	13
Pre-Test.....	13
Concepts and Discussion.....	14
Command Examples.....	14
Guided Learning Activity	15
Map the life of a change	15
Self-Test	15
Performance Task.....	15
Performance Task: Map the life of a change	15
Post-Test	15
Suggested Readings.....	16
References	16
Unit 2: Setting Up Git and GitHub	17
Overview.....	17
Introduction and Rationale	17
Learning Objectives	17
Learner Instructions	17
Materials	17

Pre-Test.....	17
Concepts and Discussion.....	18
Command Examples.....	18
Guided Learning Activity	19
Configure and verify the development identity	19
Self-Test	19
Performance Task.....	19
Performance Task: Configure and verify the development identity	19
Post-Test	19
Suggested Readings.....	20
References	20
Unit 3: Creating the First Repository	21
Overview.....	21
Introduction and Rationale	21
Learning Objectives	21
Learner Instructions	21
Materials	21
Pre-Test.....	21
Concepts and Discussion.....	22
Command Examples.....	22
Guided Learning Activity	23
Create and publish student-portfolio	23
Self-Test	23
Performance Task.....	23
Performance Task: Create and publish student-portfolio	23
Post-Test	24
Suggested Readings.....	24
References	24
Unit 4: Essential Git Commands.....	25
Overview.....	25
Introduction and Rationale	25
Learning Objectives	25
Learner Instructions	25
Materials	25
Pre-Test.....	25
Concepts and Discussion.....	26
Command Examples.....	26
Guided Learning Activity	27
Build a three-commit learning history.....	27

Self-Test	27
Performance Task.....	27
Performance Task: Build a three-commit learning history	27
Post-Test	28
Suggested Readings.....	28
References	28
Unit 5: Branching and Merging	29
Overview.....	29
Introduction and Rationale	29
Learning Objectives	29
Learner Instructions	29
Materials	29
Pre-Test.....	29
Concepts and Discussion.....	30
Command Examples.....	30
Guided Learning Activity	31
Develop and merge an About page	31
Self-Test	31
Performance Task.....	31
Performance Task: Develop and merge an About page	31
Post-Test	31
Suggested Readings.....	32
References	32
Unit 6: Collaborating with Teams	33
Overview.....	33
Introduction and Rationale	33
Learning Objectives	33
Learner Instructions	33
Materials	33
Pre-Test.....	33
Concepts and Discussion.....	34
Command Examples.....	34
Guided Learning Activity	35
Complete a peer-reviewed pull request	35
Self-Test	35
Performance Task.....	35
Performance Task: Complete a peer-reviewed pull request.....	35
Post-Test	35
Suggested Readings.....	36

References	36
Unit 7: Managing Issues and Projects	37
Overview	37
Introduction and Rationale	37
Learning Objectives	37
Learner Instructions	37
Materials	37
Pre-Test	37
Concepts and Discussion	38
Command Examples	38
Guided Learning Activity	38
Track one portfolio improvement from backlog to done	38
Self-Test	39
Performance Task	39
Performance Task: Track one portfolio improvement from backlog to done	39
Post-Test	39
Suggested Readings	39
References	39
Unit 8: Repository and Workflow Best Practices	40
Overview	40
Introduction and Rationale	40
Learning Objectives	40
Learner Instructions	40
Materials	40
Pre-Test	40
Concepts and Discussion	41
Command Examples	41
Guided Learning Activity	42
Conduct a repository quality and secret-safety audit	42
Self-Test	42
Performance Task	42
Performance Task: Conduct a repository quality and secret-safety audit	42
Post-Test	42
Suggested Readings	43
References	43
Unit 9: GitHub Actions and Automation	44
Overview	44
Introduction and Rationale	44
Learning Objectives	44

Learner Instructions	44
Materials	44
Pre-Test.....	44
Concepts and Discussion.....	45
Command Examples.....	46
Guided Learning Activity	46
Create a portfolio validation workflow	46
Self-Test	46
Performance Task.....	46
Performance Task: Create a portfolio validation workflow	46
Post-Test	47
Suggested Readings.....	47
References	47
Unit 10: GitHub Pages and Static-Site Publishing.....	48
Overview.....	48
Introduction and Rationale	48
Learning Objectives	48
Learner Instructions	48
Materials	48
Pre-Test.....	48
Concepts and Discussion.....	49
Command Examples.....	49
Guided Learning Activity	50
Publish the student portfolio	50
Self-Test	50
Performance Task.....	50
Performance Task: Publish the student portfolio.....	50
Post-Test	50
Suggested Readings.....	51
References	51
Unit 11: Security and Advanced GitHub Features.....	52
Overview.....	52
Introduction and Rationale	52
Learning Objectives	52
Learner Instructions	52
Materials	52
Pre-Test.....	52
Concepts and Discussion.....	53
Command Examples.....	53

Guided Learning Activity	54
Audit repository security and advanced-feature readiness.....	54
Self-Test	54
Performance Task.....	54
Performance Task: Audit repository security and advanced-feature readiness ..	54
Post-Test	54
Suggested Readings.....	55
References	55
Unit 12: Contributing to Open Source Projects	56
Overview.....	56
Introduction and Rationale	56
Learning Objectives	56
Learner Instructions	56
Materials	56
Pre-Test.....	56
Concepts and Discussion.....	57
Command Examples.....	57
Guided Learning Activity	58
Prepare an open-source contribution.....	58
Self-Test	58
Performance Task.....	58
Performance Task: Prepare an open-source contribution.....	58
Post-Test	58
Suggested Readings.....	59
References	59
Unit 13: Troubleshooting and Recovery.....	60
Overview.....	60
Introduction and Rationale	60
Learning Objectives	60
Learner Instructions	60
Materials	60
Pre-Test.....	60
Concepts and Discussion.....	61
Command Examples.....	61
Guided Learning Activity	62
Diagnose and recover four controlled failures.....	62
Self-Test	62
Performance Task.....	62
Performance Task: Diagnose and recover four controlled failures	62

Post-Test	63
Suggested Readings.....	63
References	63
Capstone Project.....	64
Capstone: Publish a Reviewed Student Portfolio	64
Assessment Rubrics.....	64
Unit Performance Task Rubric	64
Pull Request Rubric	64
Capstone Rubric	65
Answer Key	66
Unit 1: Introduction to Git and GitHub.....	66
Unit 2: Setting Up Git and GitHub	67
Unit 3: Creating the First Repository	69
Unit 4: Essential Git Commands	70
Unit 5: Branching and Merging.....	72
Unit 6: Collaborating with Teams	73
Unit 7: Managing Issues and Projects	75
Unit 8: Repository and Workflow Best Practices	76
Unit 9: GitHub Actions and Automation	78
Unit 10: GitHub Pages and Static-Site Publishing	79
Unit 11: Security and Advanced GitHub Features	81
Unit 12: Contributing to Open Source Projects.....	82
Unit 13: Troubleshooting and Recovery	84
Git Command Cheat Sheet.....	85
References.....	86
About the Author	87

List of Figures

- Figure 1. From a working file to shared repository history
- Figure 2. A secure Git and GitHub setup path
- Figure 3. Two paths to the first published repository
- Figure 4. The inspect stage commit and synchronize cycle
- Figure 5. Feature branching merging and conflict resolution
- Figure 6. The collaborative pull request cycle
- Figure 7. Tracking work from backlog to completion
- Figure 8. Elements of a healthy repository
- Figure 9. A continuous integration validation pipeline
- Figure 10. Publishing a static site with GitHub Pages

Figure 11. Layered protection for an account and repository

Figure 12. An open source contribution journey

Figure 13. A safe first troubleshooting decision path

Glossary

Term	Definition
Action	A reusable task that runs as a step in a GitHub Actions workflow.
Authentication	The process of proving identity to GitHub or another remote service.
Branch	A movable name that points to a line of commits.
Clone	A local copy of a repository that includes its history and remote connection.
Code review	A structured examination of proposed changes before they are merged.
Commit	A recorded snapshot of staged changes with author, time, and message metadata.
Commit hash	A unique identifier for a commit object.
Conflict	A situation in which Git cannot automatically combine competing changes.
Continuous integration	Automated validation of changes whenever selected repository events occur.
Default branch	The branch GitHub presents as the repository's primary line of development.
Detached HEAD	A state in which HEAD points directly to a commit rather than a branch.
Diff	A line-by-line representation of changes between file states.
Distributed version control	A model in which each clone contains the project's complete history.

Term	Definition
Fetch	The operation that downloads remote objects and references without merging them.
Fork	A GitHub-hosted copy of another user's repository under a different account.
Git	A distributed version control system for tracking changes to files.
GitHub	A hosted collaboration platform built around Git repositories.
HEAD	Git's reference to the commit or branch currently checked out.
Issue	A GitHub record used to discuss and track work, bugs, or proposals.
Job	A set of GitHub Actions steps executed by the same runner.
Merge	The operation that combines histories from two branches.
Milestone	A grouping of issues and pull requests toward a target outcome or date.
Origin	The conventional name for the primary remote created during cloning.
Personal access token	A revocable credential used in place of a password for supported operations.
Project	A GitHub planning view for organizing and tracking issues and pull requests.
Pull	A fetch followed by integration into the current branch.
Pull request	A GitHub proposal to review and merge changes from one branch into another.
Push	The operation that uploads local commits and updates remote references.

Term	Definition
Remote	A named connection to another copy of a repository.
Repository	A project directory together with its Git objects, references, and configuration.
Restore	A Git command that restores working-tree or staged file content from another state.
Runner	A machine that executes jobs in a GitHub Actions workflow.
Staging area	The index that holds the exact content planned for the next commit.
Tag	A durable name for a specific commit, often used for releases.
Workflow	A YAML-defined automated process stored in <code>`.github/workflows`</code> .
Working tree	The files currently checked out for viewing and editing.

Semester Roadmap

Unit	Topic	Time	Output	Assessment
1	Introduction to Git and GitHub	2 hours	Version-control concept map	Post-test and performance rubric
2	Setting Up Git and GitHub	2 hours	Verified Git identity and authentication	Post-test and performance rubric
3	Creating the First Repository	2 hours	Repository with initial commit	Post-test and performance rubric
4	Essential Git Commands	2 hours	Three-commit project history	Post-test and performance rubric

Unit	Topic	Time	Output	Assessment
5	Branching and Merging	3 hours	Merged feature branch and resolved conflict	Post-test and performance rubric
6	Collaborating with Teams	3 hours	Reviewed pull request	Post-test and performance rubric
7	Managing Issues and Projects	2 hours	Tracked issue and project item	Post-test and performance rubric
8	Repository and Workflow Best Practices	2 hours	Repository quality audit	Post-test and performance rubric
9	GitHub Actions and Automation	3 hours	Passing validation workflow	Post-test and performance rubric
10	GitHub Pages and Static-Site Publishing	3 hours	Published portfolio site	Post-test and performance rubric
11	Security and Advanced GitHub Features	2 hours	Repository security review	Post-test and performance rubric
12	Contributing to Open Source Projects	2 hours	Contribution plan or documentation pull request	Post-test and performance rubric
13	Troubleshooting and Recovery	3 hours	Troubleshooting evidence log	Post-test and performance rubric

Unit 1: Introduction to Git and GitHub

Time allotment: 2 hours

Overview

This unit builds the mental model used throughout the module: files begin in a working tree, selected versions enter the staging area, commits form local history, and a remote repository supports sharing.

Introduction and Rationale

Students who understand the states of a change can interpret Git messages, choose commands deliberately, and recover from mistakes instead of memorizing unexplained command sequences.

Learning Objectives

1. Differentiate Git, GitHub, and an ordinary file-storage service.
2. Identify a repository, commit, branch, clone, and remote in a development workflow.
3. Trace a file change from the working tree through the staging area and local history to a remote.
4. Explain how version history supports accountability, collaboration, and recovery.

Learner Instructions

- Complete the pre-test without consulting notes.
- Use the student-portfolio example when drawing the workflow.
- Describe each arrow in the concept map with an action and resulting state.

Materials

- Computer or paper for diagramming
- Web browser
- This module

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What problem does version control solve?
2. How does Git differ from GitHub?
3. What does a commit contain?
4. What is the staging area?
5. What is a branch?

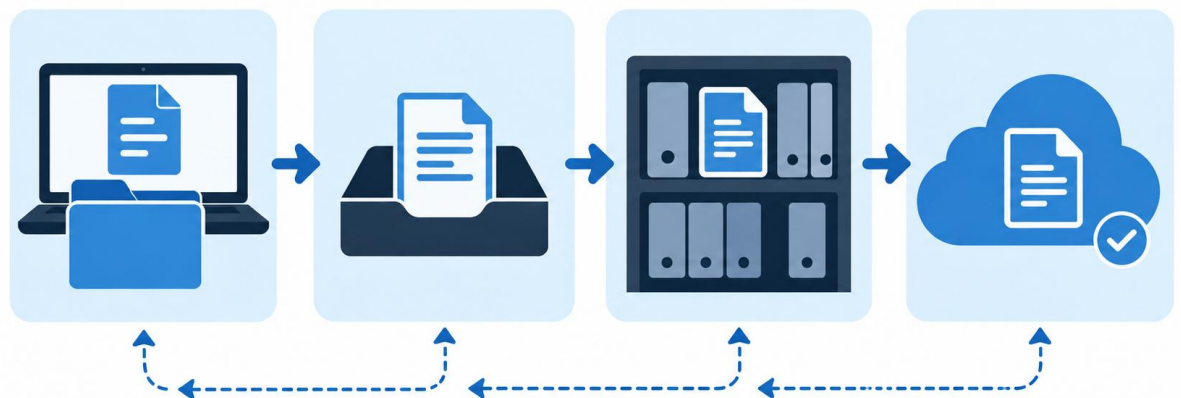


Figure 1. From a working file to shared repository history

Source: Original instructional illustration generated for this module.

Concepts and Discussion

Version control records meaningful states of a project over time. Unlike a sequence of folders named final, final-2, and final-revised, a repository preserves ordered snapshots with messages, authorship, and relationships among changes.

Git is the version control system that operates on a local computer. GitHub hosts Git repositories and adds pull requests, issues, projects, automation, access controls, and web publishing. Git can be used without GitHub, and GitHub also provides features beyond Git itself.

A repository is the project working tree plus the hidden Git database that stores objects and references. A commit records the exact staged content, a message, author information, time, and a link to its parent commit or commits.

The staging area is a deliberate checkpoint between editing and committing. It lets a student choose which completed changes belong together. Unstaged work remains in the working tree; staged content is proposed for the next commit; committed content belongs to local history.

A branch is a movable name pointing to a commit. Branches let people develop changes without immediately altering the default branch. A remote is a named connection to another repository; ``origin`` is the conventional name assigned when cloning.

Cloning creates a local repository with history and a configured remote. Pushing transfers local commits to a remote. Fetching downloads remote history without integrating it, while pulling fetches and then integrates according to configuration.

Command Examples

```
git status
```

Reports the current branch and the state of working-tree and staged changes.

```
git log --oneline
```

Shows a compact view of recorded commits.

Guided Learning Activity

Map the life of a change

1. Draw four labeled zones: working tree, staging area, local repository, and GitHub remote.
2. Place a README.md change in the working tree and show how add, commit, and push move its selected state.
3. Add reverse arrows for fetch and pull, and explain that fetch alone does not edit the current working files.
4. Compare the map with a partner and correct any arrow that skips a required state.

Required evidence: An annotated concept map with four states and accurate action arrows

Safety and privacy: Use a fictional project name and omit account credentials from the diagram.

Self-Test

1. What does cloning create?
2. Why use a remote repository?
3. What is the working tree?
4. What is the difference between fetch and pull?
5. Why are descriptive commit messages useful?

Performance Task

Performance Task: Map the life of a change

1. Draw four labeled zones: working tree, staging area, local repository, and GitHub remote.
2. Place a README.md change in the working tree and show how add, commit, and push move its selected state.
3. Add reverse arrows for fetch and pull, and explain that fetch alone does not edit the current working files.
4. Compare the map with a partner and correct any arrow that skips a required state.

Required evidence: An annotated concept map with four states and accurate action arrows

Safety and privacy: Use a fictional project name and omit account credentials from the diagram.

Post-Test

1. What problem does version control solve?
2. How does Git differ from GitHub?
3. What does a commit contain?
4. What is the staging area?
5. What is a branch?
6. What does cloning create?
7. Why use a remote repository?

8. What is the working tree?
9. What is the difference between fetch and pull?
10. Why are descriptive commit messages useful?

Suggested Readings

- Git reference: Getting Started
- GitHub Docs: About Git

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 2: Setting Up Git and GitHub

Time allotment: 2 hours

Overview

This unit prepares a secure working environment by installing Git, configuring commit identity, creating a GitHub account, and selecting an authentication method.

Introduction and Rationale

A correct setup prevents unattributed commits, failed pushes, and unsafe credential handling during later activities.

Learning Objectives

1. Install Git and verify the installed version.
2. Configure and inspect the name and email recorded in commits.
3. Compare HTTPS and SSH authentication for student use.
4. Verify GitHub access without exposing a password, token, or private key.

Learner Instructions

- Follow the installation path approved for the laboratory computer.
- Use the name and email directed by the instructor.
- Redact personal information before submitting terminal output.

Materials

- Computer with installation permission
- GitHub account
- Internet connection

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. How do you verify a Git installation?
2. What does user.name control?
3. What does user.email control?
4. Is the Git email a login password?
5. What is the difference between global and local configuration?

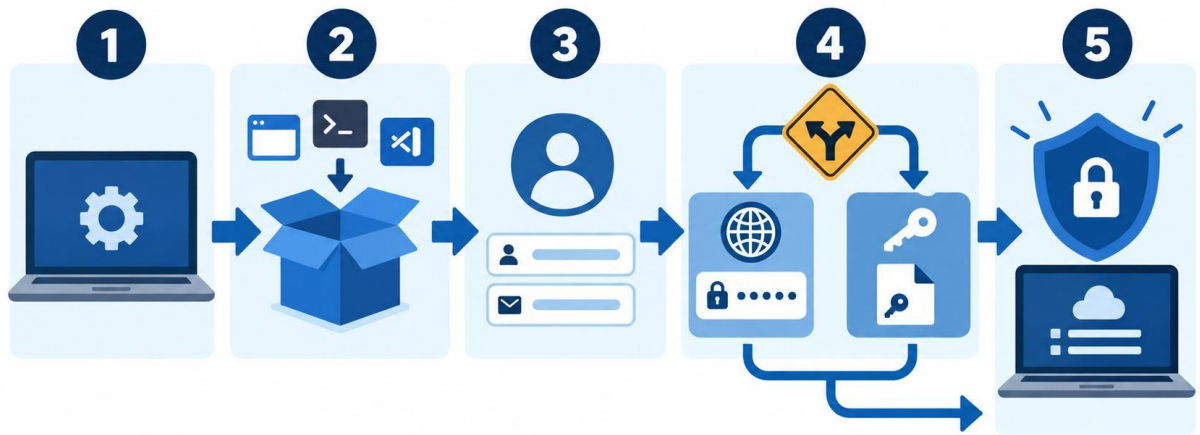


Figure 2. A secure Git and GitHub setup path

Source: Original instructional illustration generated for this module.

Concepts and Discussion

Git is available for Windows, macOS, and Linux. Installation methods differ, but verification is consistent: the ``git --version`` command should report an installed release rather than an unrecognized command.

Git records a user name and email in each commit. These values identify authorship; they are not login credentials. A student may use a verified GitHub email or a GitHub-provided no-reply address when privacy is important.

Global configuration applies to repositories for the current operating-system account. Repository-local configuration can override it when a course or workplace requires a different identity. Students should inspect effective values before the first commit.

HTTPS and SSH are both valid remote protocols. HTTPS commonly uses a credential manager or personal access token. SSH uses a public/private key pair. The public key may be registered with GitHub; the private key remains only on the student's device.

Authentication proves access to GitHub, while authorization determines what the account may do. A successful account login does not automatically permit writes to every repository.

Security evidence should demonstrate a successful connection without revealing credentials. A screenshot must exclude password fields, token values, recovery codes, private-key content, and unrelated personal information.

Command Examples

```
git --version
```

Verifies that Git is installed and available on PATH.

```
git config --global user.name "Student Name"
```

Sets the author name for new commits.

```
git config --global user.email "student@example.edu"
```

Sets the author email for new commits.

```
git config --global --list
```

Displays global configuration for review; redact private values before submission.

```
ssh -T git@github.com
```

Tests SSH authentication after a public key is registered; no repository data is changed.

Guided Learning Activity

Configure and verify the development identity

1. Run `git --version` and record the reported version.
2. Configure the instructor-approved name and email.
3. Inspect the effective configuration and correct typographical errors.
4. Choose HTTPS with a credential manager or SSH with a protected private key.
5. Verify access and submit only redacted evidence of success.

Required evidence: Git version, redacted identity configuration, and verified GitHub access

Safety and privacy: Never submit a password, personal access token, recovery code, or private key. Submit only redacted verification output.

Self-Test

1. What is safe to upload from an SSH key pair?
2. How does HTTPS commonly authenticate Git operations?
3. What should be redacted from setup evidence?
4. Why verify identity before the first commit?
5. What does a successful authentication test prove?

Performance Task

Performance Task: Configure and verify the development identity

1. Run `git --version` and record the reported version.
2. Configure the instructor-approved name and email.
3. Inspect the effective configuration and correct typographical errors.
4. Choose HTTPS with a credential manager or SSH with a protected private key.
5. Verify access and submit only redacted evidence of success.

Required evidence: Git version, redacted identity configuration, and verified GitHub access

Safety and privacy: Never submit a password, personal access token, recovery code, or private key. Submit only redacted verification output.

Post-Test

1. How do you verify a Git installation?
2. What does `user.name` control?
3. What does `user.email` control?
4. Is the Git email a login password?
5. What is the difference between global and local configuration?
6. What is safe to upload from an SSH key pair?
7. How does HTTPS commonly authenticate Git operations?

8. What should be redacted from setup evidence?
9. Why verify identity before the first commit?
10. What does a successful authentication test prove?

Suggested Readings

- Git reference: git-config
- GitHub Docs: Connecting to GitHub with SSH

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 3: Creating the First Repository

Time allotment: 2 hours

Overview

Students create the continuing student-portfolio repository, inspect its remote connection, record an initial README, and publish the first commit.

Introduction and Rationale

A small, well-documented repository provides a safe workspace for every later branching, collaboration, automation, and publishing exercise.

Learning Objectives

1. Create and clone a GitHub repository using a beginner-safe workflow.
2. Initialize a local repository and connect it to a remote when required.
3. Record an initial README in a focused commit.
4. Verify the remote and publish the default branch.

Learner Instructions

- Use the repository name student-portfolio unless the instructor assigns another name.
- Do not initialize the same directory twice.
- Check status after every major step.

Materials

- Configured Git installation
- GitHub account
- Text editor

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What does git clone create?
2. What does git init create?
3. Why inspect git remote -v?
4. What belongs in the first README?
5. Why check status before committing?

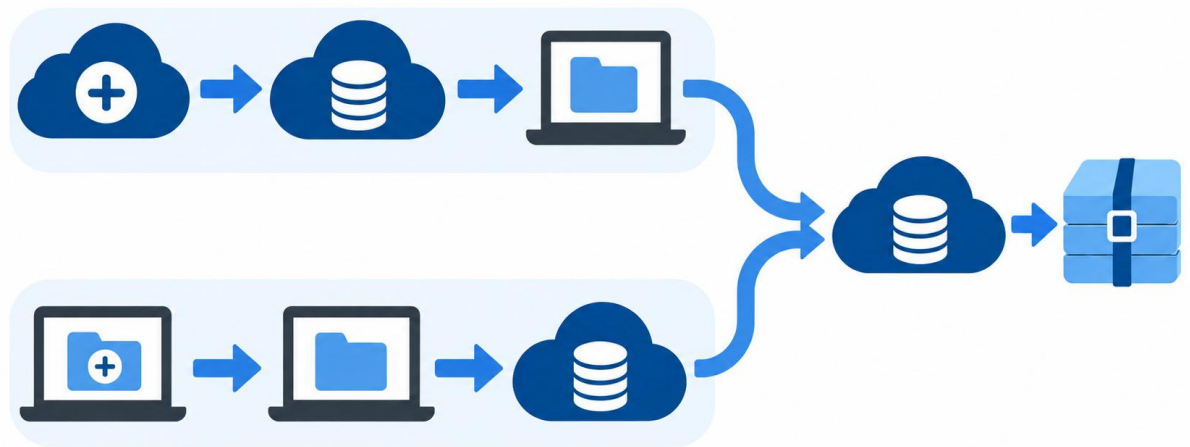


Figure 3. Two paths to the first published repository

Source: Original instructional illustration generated for this module.

Concepts and Discussion

A repository can begin remotely or locally. For beginners, creating a GitHub repository with a README and then cloning it reduces remote-configuration mistakes. Existing local work can instead be initialized and connected to an empty remote.

Cloning creates a new project directory, copies Git history, checks out the default branch, and creates the `origin` remote. Students should not clone into a directory that already contains unrelated work.

Initializing with `git init` creates the local Git database but does not create a GitHub repository or upload files. A remote must be added deliberately, and the local branch name should match the intended default branch.

A README explains the project's purpose, current contents, and how another reader can understand or use it. The initial README should be brief, accurate, and free of private student information.

The first commit establishes a meaningful baseline. `git status` should be read before staging and again after committing so the student observes how the repository state changes.

The upstream link created by `git push -u origin main` lets later `git push` and `git pull` commands infer the associated remote branch. The command succeeds only when authentication and permissions are valid.

Command Examples

```
git clone https://github.com/ACCOUNT/student-portfolio.git
```

Creates a local copy and configures origin. Replace ACCOUNT with the actual account name.

```
git init
```

Initializes Git in the current directory for the alternate local-first path.

```
git branch -M main
```

Names the current branch main before the first push in the local-first path.

```
git remote add origin https://github.com/ACCOUNT/student-portfolio.git
```

Connects a local repository to its GitHub remote.

```
git remote -v
```

Displays fetch and push URLs for configured remotes.

```
git add README.md
```

Stages the README content for the next commit.

```
git commit -m "docs: add initial project overview"
```

Records the initial documented state.

```
git push -u origin main
```

Publishes main and records its upstream remote branch.

Guided Learning Activity

Create and publish student-portfolio

1. Create the repository on GitHub with the visibility directed by the instructor and include a README.
2. Clone the repository, enter its directory, and inspect git status and git remote -v.
3. Add a purpose statement and learning-goals section to README.md.
4. Stage only README.md, commit with a descriptive message, and push.
5. Confirm on GitHub that the commit and README are visible.

Required evidence: Repository URL or approved screenshot plus redacted git log --oneline output

Safety and privacy: Do not publish student numbers, home addresses, passwords, tokens, private keys, or confidential course data.

Self-Test

1. What does origin mean?
2. What does -u do on the first push?
3. Why might a push be rejected?
4. What is the safer beginner repository-start path?
5. What should repository evidence exclude?

Performance Task

Performance Task: Create and publish student-portfolio

1. Create the repository on GitHub with the visibility directed by the instructor and include a README.
2. Clone the repository, enter its directory, and inspect git status and git remote -v.
3. Add a purpose statement and learning-goals section to README.md.
4. Stage only README.md, commit with a descriptive message, and push.
5. Confirm on GitHub that the commit and README are visible.

Required evidence: Repository URL or approved screenshot plus redacted git log --oneline output

Safety and privacy: Do not publish student numbers, home addresses, passwords, tokens, private keys, or confidential course data.

Post-Test

1. What does git clone create?
2. What does git init create?
3. Why inspect git remote -v?
4. What belongs in the first README?
5. Why check status before committing?
6. What does origin mean?
7. What does -u do on the first push?
8. Why might a push be rejected?
9. What is the safer beginner repository-start path?
10. What should repository evidence exclude?

Suggested Readings

- GitHub Docs: Quickstart for repositories
- Git reference: git-clone and git-init

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 4: Essential Git Commands

Time allotment: 2 hours

Overview

This unit turns individual commands into a repeatable edit, inspect, stage, review, commit, and synchronize workflow.

Introduction and Rationale

A disciplined cycle produces focused commits, reduces accidental changes, and gives students reliable checkpoints for later collaboration.

Learning Objectives

1. Inspect working-tree and staging-area state before recording changes.
2. Stage selected content and review its diff.
3. Create focused commits with descriptive messages.
4. Synchronize local and remote history using fetch, pull, and push deliberately.

Learner Instructions

- Run status before and after each state-changing command.
- Commit one coherent change at a time.
- Read command output instead of assuming success.

Materials

- student-portfolio repository
- Text editor
- Terminal

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What does git status report?
2. What does git diff show by default?
3. What does git diff --staged show?
4. What does git add do?
5. What does git commit do?

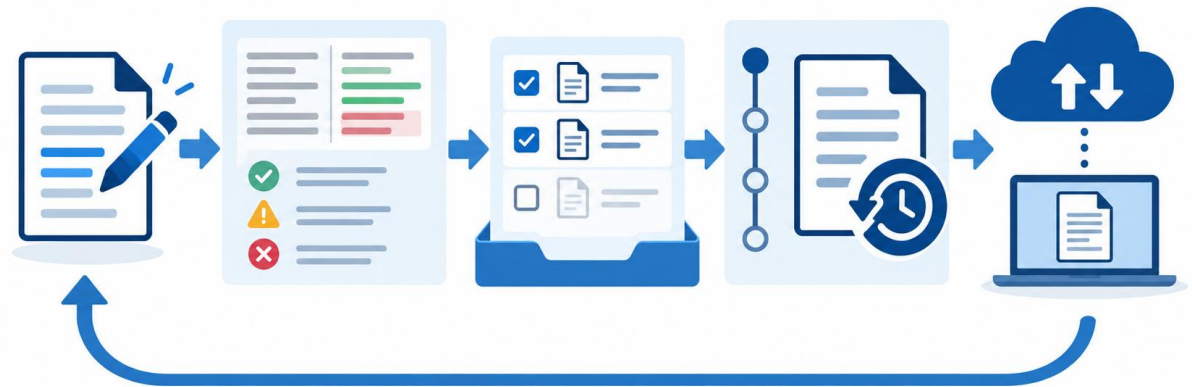


Figure 4. The inspect stage commit and synchronize cycle

Source: Original instructional illustration generated for this module.

Concepts and Discussion

`git status` is the primary orientation command. It reports the current branch, upstream relationship, staged changes, unstaged changes, and untracked files. A clean status means the working tree matches the current commit.

`git diff` shows unstaged changes by default. `git diff --staged` shows the exact content proposed for the next commit. Reviewing both prevents unrelated edits or debugging output from entering history.

`git add` copies selected file content into the staging area. It does not permanently save a version and it does not upload anything. Editing a file after staging creates a staged version and a newer unstaged version, which status displays separately.

`git commit` creates a local history entry from staged content. A strong message begins with a concise action and explains one coherent purpose, such as `docs: add skills section to portfolio`.

`git log --oneline --graph --decorate` reveals commit order, branch labels, and merges. Students should use the history to verify that a requested commit exists rather than relying only on the final files.

`git fetch` updates remote-tracking information without changing the current branch. `git pull` fetches and integrates. `git push` publishes local commits. Inspecting status and history before synchronization makes unexpected divergence easier to diagnose.

Command Examples

```
git status
```

Orient before choosing the next command.

```
git diff
```

Review unstaged line changes.

```
git add README.md
```

Stage one file deliberately.

```
git diff --staged
```

Review the exact staged content.

```
git commit -m "docs: add portfolio skills"
```

Record one coherent change in local history.

```
git log --oneline --graph --decorate
```

Inspect compact history and branch labels.

```
git fetch origin
```

Download remote history without integrating it.

```
git pull --ff-only
```

Update only when a fast-forward is possible, avoiding an accidental merge commit.

```
git push
```

Publish commits to the configured upstream branch.

Guided Learning Activity

Build a three-commit learning history

1. Add an About section to README.md, inspect the diff, stage it, review the staged diff, and commit.
2. Add a Skills section and record it as a second focused commit.
3. Add a Contact section using only instructor-approved public information and record a third focused commit.
4. Inspect the graph, push the commits, and verify that status reports a clean working tree.

Required evidence: A three-commit git log and a clean git status after push

Safety and privacy: Use only public contact information approved for the activity; do not commit private addresses, passwords, tokens, or keys.

Self-Test

1. Why create focused commits?
2. What does git fetch change?
3. How does pull differ from fetch?
4. When is git push used?
5. What does a clean working tree mean?

Performance Task

Performance Task: Build a three-commit learning history

1. Add an About section to README.md, inspect the diff, stage it, review the staged diff, and commit.
2. Add a Skills section and record it as a second focused commit.
3. Add a Contact section using only instructor-approved public information and record a third focused commit.
4. Inspect the graph, push the commits, and verify that status reports a clean working tree.

Required evidence: A three-commit git log and a clean git status after push

Safety and privacy: Use only public contact information approved for the activity; do not commit private addresses, passwords, tokens, or keys.

Post-Test

1. What does git status report?
2. What does git diff show by default?
3. What does git diff --staged show?
4. What does git add do?
5. What does git commit do?
6. Why create focused commits?
7. What does git fetch change?
8. How does pull differ from fetch?
9. When is git push used?
10. What does a clean working tree mean?

Suggested Readings

- Git reference: Everyday Git
- GitHub Docs: Pushing commits to a remote repository

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 5: Branching and Merging

Time allotment: 3 hours

Overview

Students isolate work on a feature branch, inspect branch history, merge a completed change, and resolve a controlled conflict.

Introduction and Rationale

Branches let teams develop and review changes without destabilizing the shared default branch.

Learning Objectives

1. Create and switch branches safely.
2. Explain divergence and merge outcomes.
3. Merge a focused feature into main.
4. Resolve a text conflict without discarding valid work.

Learner Instructions

- Begin with a clean working tree.
- Use the assigned feature branch name.
- Read conflict markers before editing.

Materials

- student-portfolio repository
- Text editor
- Terminal

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What does a branch point to?
2. What does `git switch -c` do?
3. Why confirm the active branch before editing?
4. When is a merge fast-forward?
5. What causes a merge conflict?

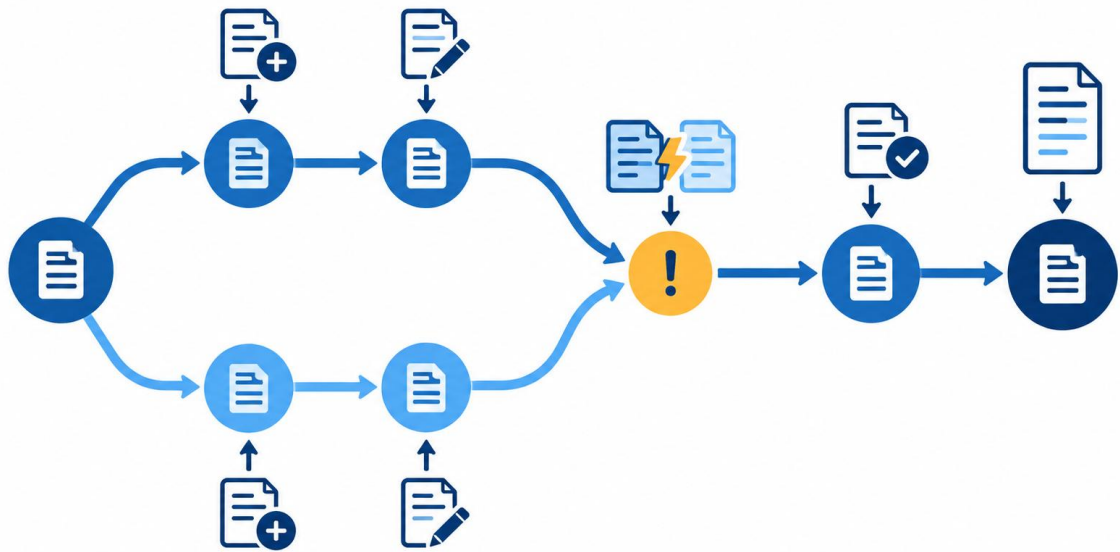


Figure 5. Feature branching merging and conflict resolution

Source: Original instructional illustration generated for this module.

Concepts and Discussion

A branch is a movable reference to the latest commit in a line of development. Creating a branch is inexpensive because Git stores commits once and moves names as new commits are added.

`git switch -c` creates a branch from the current commit and checks it out. The working tree then reflects that branch, and new commits advance it rather than main.

A fast-forward merge moves the target branch name forward when no competing commits exist. A three-way merge combines divergent histories and records a merge commit when necessary.

A conflict occurs when Git cannot safely decide how competing edits should be combined. Conflict markers show the current and incoming content; they are instructions for a human decision, not content to commit.

Conflict resolution means constructing the correct final file, removing all markers, testing or reviewing it, staging the resolved file, and completing the merge. Branch deletion happens only after the work is safely integrated.

Command Examples

```
git switch -c feature/about-page
```

Creates and checks out a focused feature branch.

```
git branch --show-current
```

Confirms the active branch before editing.

```
git log --oneline --graph --decorate --all
```

Shows branch divergence and merge structure.

```
git switch main
```

Returns to the target branch.

```
git merge feature/about-page
```

Integrates the completed feature into main.

```
git branch -d feature/about-page
```

Deletes the integrated local branch after verification.

Guided Learning Activity

Develop and merge an About page

1. Create feature/about-page from an up-to-date main branch and add about.html.
2. Commit the page on the feature branch and inspect the graph.
3. Create a controlled conflicting edit in an instructor-provided practice file on both branches.
4. Merge, inspect the conflict markers, preserve the intended content from both sides, and remove every marker.
5. Stage the resolved file, complete the merge, inspect history, and delete the feature branch after verification.

Required evidence: A merged feature, resolved conflict file, and graph showing the integrated history

Safety and privacy: Resolve conflicts only in the practice repository; make a backup before experimenting with history changes.

Self-Test

1. What do conflict markers represent?
2. What is the goal of conflict resolution?
3. How is a resolved file recorded?
4. When is local branch deletion safe?
5. Why inspect the commit graph?

Performance Task

Performance Task: Develop and merge an About page

1. Create feature/about-page from an up-to-date main branch and add about.html.
2. Commit the page on the feature branch and inspect the graph.
3. Create a controlled conflicting edit in an instructor-provided practice file on both branches.
4. Merge, inspect the conflict markers, preserve the intended content from both sides, and remove every marker.
5. Stage the resolved file, complete the merge, inspect history, and delete the feature branch after verification.

Required evidence: A merged feature, resolved conflict file, and graph showing the integrated history

Safety and privacy: Resolve conflicts only in the practice repository; make a backup before experimenting with history changes.

Post-Test

1. What does a branch point to?
2. What does git switch -c do?
3. Why confirm the active branch before editing?
4. When is a merge fast-forward?
5. What causes a merge conflict?

6. What do conflict markers represent?
7. What is the goal of conflict resolution?
8. How is a resolved file recorded?
9. When is local branch deletion safe?
10. Why inspect the commit graph?

Suggested Readings

- Git reference: git-switch
- Git reference: git-merge

References

Git. (2026). Git reference. <https://git-scm.com/docs>

Unit 6: Collaborating with Teams

Time allotment: 3 hours

Overview

Students use branches or forks, pull requests, review comments, automated checks, and merge decisions to collaborate transparently.

Introduction and Rationale

A pull request turns code integration into a reviewable conversation with evidence, ownership, and a durable decision record.

Learning Objectives

1. Select a branch or fork workflow.
2. Open a focused pull request.
3. Review changes constructively.
4. Revise and merge after requirements are met.

Learner Instructions

- Use an instructor-assigned partner.
- Keep the pull request focused on one issue.
- Respond to review comments respectfully and with evidence.

Materials

- Shared or forked repository
- GitHub account
- Review rubric

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. When is a shared-branch workflow appropriate?
2. When is a fork workflow appropriate?
3. What should a pull-request description explain?
4. What is a pull-request diff?
5. What makes a useful review comment?



Figure 6. The collaborative pull request cycle

Source: Original instructional illustration generated for this module.

Concepts and Discussion

Collaborators with write access commonly create branches in one repository. External contributors commonly fork, clone the fork, and open a pull request back to the original repository.

A pull request identifies a source branch, target branch, purpose, related issue, and visible diff. A good description explains why the change is needed, what changed, and how it was checked.

Reviewers examine correctness, scope, clarity, safety, and test evidence. Comments should identify a specific observation and desired outcome rather than judging the contributor.

Requested changes are part of collaboration. New commits pushed to the source branch update the same pull request, preserving the discussion and review history.

Merging should occur only after required reviews and checks pass. Merge, squash, and rebase strategies produce different histories; the repository's contribution policy determines which is appropriate.

Command Examples

```
git fetch origin
```

Updates remote-tracking information before starting work.

```
git switch -c feature/about-page origin/main
```

Creates a feature branch from current remote main.

```
git push -u origin feature/about-page
```

Publishes the feature branch and sets its upstream.

```
git log origin/main..HEAD --oneline
```

Lists commits proposed by the current branch.

```
git diff origin/main...HEAD
```

Reviews the complete proposed change before opening a pull request.

Guided Learning Activity

Complete a peer-reviewed pull request

1. Choose one assigned issue and create a focused branch.
2. Make and review small commits, then push the branch.
3. Open a pull request that links the issue and states validation evidence.
4. Review a partner's pull request using the rubric and request one concrete improvement when needed.
5. Address feedback, obtain approval, confirm checks, and merge using the approved strategy.

Required evidence: A merged pull request with linked issue, peer review, revision evidence, and passing checks

Safety and privacy: Use only course repositories and never paste credentials, private student data, or secrets into issues, commits, reviews, or pull requests.

Self-Test

1. How is a pull request updated?
2. Why link an issue?
3. What must happen before merge?
4. What does squash merge do?
5. Why preserve review history?

Performance Task

Performance Task: Complete a peer-reviewed pull request

1. Choose one assigned issue and create a focused branch.
2. Make and review small commits, then push the branch.
3. Open a pull request that links the issue and states validation evidence.
4. Review a partner's pull request using the rubric and request one concrete improvement when needed.
5. Address feedback, obtain approval, confirm checks, and merge using the approved strategy.

Required evidence: A merged pull request with linked issue, peer review, revision evidence, and passing checks

Safety and privacy: Use only course repositories and never paste credentials, private student data, or secrets into issues, commits, reviews, or pull requests.

Post-Test

1. When is a shared-branch workflow appropriate?
2. When is a fork workflow appropriate?
3. What should a pull-request description explain?
4. What is a pull-request diff?
5. What makes a useful review comment?
6. How is a pull request updated?
7. Why link an issue?
8. What must happen before merge?
9. What does squash merge do?

10. Why preserve review history?

Suggested Readings

- GitHub Docs: About pull requests
- GitHub Docs: Reviewing proposed changes

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Git. (2026). Git reference. <https://git-scm.com/docs>

Unit 7: Managing Issues and Projects

Time allotment: 2 hours

Overview

Students turn a need into a well-scoped issue, classify and assign it, plan it on a project board, and close it with evidence.

Introduction and Rationale

Visible work tracking helps teams agree on scope, ownership, status, and completion before code changes begin.

Learning Objectives

1. Write an actionable issue.
2. Use labels, assignees, and milestones meaningfully.
3. Track work through a project workflow.
4. Link implementation evidence to completion.

Learner Instructions

- Describe an observable need.
- Write verifiable acceptance criteria.
- Update status when the work state changes.

Materials

- Course GitHub repository
- GitHub Projects access
- Issue template

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What makes an issue actionable?
2. What should an issue title communicate?
3. What is the role of labels?
4. What does an assignee represent?
5. What does a milestone group?



Figure 7. Tracking work from backlog to completion

Source: Original instructional illustration generated for this module.

Concepts and Discussion

An issue is a durable discussion and tracking record. A useful title names the outcome or problem; the body provides context, reproduction details when relevant, acceptance criteria, and supporting evidence.

Labels classify work by type, area, priority, or status. Too many overlapping labels reduce meaning, so teams should use a small documented vocabulary.

An assignee owns the next action, while a milestone groups work toward a larger target. Assignment communicates responsibility but does not replace discussion or acceptance criteria.

A project view organizes issues and pull requests through states such as backlog, ready, in progress, review, and done. The board should reflect reality rather than become a second disconnected task list.

Closing keywords in a pull-request description can close a linked issue after merge. The issue should contain final evidence or a link so another person can verify why it is complete.

Command Examples

```
gh issue list
```

Optional GitHub CLI view of open issues.

```
gh issue view 1
```

Optional detailed issue view; replace 1 with the assigned issue number.

Guided Learning Activity

Track one portfolio improvement from backlog to done

1. Create an issue with context and three verifiable acceptance criteria.

2. Apply one type label, assign the responsible student, and add the course milestone.
3. Add the issue to the project and move it from backlog to in progress.
4. Link the implementing pull request and move the item to review.
5. After merge, verify the acceptance criteria, close the issue, and move it to done.

Required evidence: A closed issue with label, assignee, milestone, project history, acceptance criteria, and linked implementation

Safety and privacy: Do not place grades, private feedback, credentials, or personal contact information in public issues or project fields.

Self-Test

1. What is a project view used for?
2. Why should project status reflect reality?
3. How can a pull request close an issue?
4. What evidence belongs on a completed issue?
5. What information should public issues exclude?

Performance Task

Performance Task: Track one portfolio improvement from backlog to done

1. Create an issue with context and three verifiable acceptance criteria.
2. Apply one type label, assign the responsible student, and add the course milestone.
3. Add the issue to the project and move it from backlog to in progress.
4. Link the implementing pull request and move the item to review.
5. After merge, verify the acceptance criteria, close the issue, and move it to done.

Required evidence: A closed issue with label, assignee, milestone, project history, acceptance criteria, and linked implementation

Safety and privacy: Do not place grades, private feedback, credentials, or personal contact information in public issues or project fields.

Post-Test

1. What makes an issue actionable?
2. What should an issue title communicate?
3. What is the role of labels?
4. What does an assignee represent?
5. What does a milestone group?
6. What is a project view used for?
7. Why should project status reflect reality?
8. How can a pull request close an issue?
9. What evidence belongs on a completed issue?
10. What information should public issues exclude?

Suggested Readings

- GitHub Docs: About issues
- GitHub Docs: About Projects

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 8: Repository and Workflow Best Practices

Time allotment: 2 hours

Overview

Students audit repository documentation, ignored files, commit quality, branch rules, and secret safety.

Introduction and Rationale

Consistent repository practices reduce accidental exposure, lower review cost, and help future contributors understand the project.

Learning Objectives

1. Improve repository documentation.
2. Exclude generated and sensitive files correctly.
3. Evaluate commit and branch hygiene.
4. Respond safely to an exposed secret.

Learner Instructions

- Audit before editing.
- Never create a real secret for this activity.
- Explain each recommendation with evidence.

Materials

- student-portfolio repository
- Quality checklist
- Sample unsafe repository

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What should a README establish?
2. What does .gitignore affect?
3. Why ignore .env?
4. What makes a commit focused?
5. Why protect the default branch?



Figure 8. Elements of a healthy repository

Source: Original instructional illustration generated for this module.

Concepts and Discussion

A README should answer what the project is, why it exists, how to view or use it, and how to contribute when contributions are expected. Documentation must match the current repository rather than promise unfinished features.

`.gitignore` prevents selected untracked files from being offered for commit. Common entries include `.env`, build output, dependency folders, editor state, and operating-system metadata. It does not remove files already tracked.

Small commits with purposeful messages make review and recovery easier. A commit should represent one coherent change and should not mix formatting, unrelated refactoring, and new behavior without a reason.

Protected collaboration keeps important branches stable by requiring pull requests, reviews, or checks. Students should pull or fetch before starting work and avoid rewriting history that other people may have based work on.

A committed secret remains in history even after its latest line is deleted. The correct response begins by revoking or rotating the credential, then removing it from current files, adding ignore protection, and following the repository's approved history-cleanup procedure.

Command Examples

```
git check-ignore -v .env
```

Explains which ignore rule applies to a sensitive configuration file.

```
git ls-files .env
```

Reveals whether `.env` is already tracked.

```
git log --oneline --graph --decorate --all
```

Supports a commit-history quality audit.

```
git status --ignored
```

Shows ignored paths during verification.

Guided Learning Activity

Conduct a repository quality and secret-safety audit

1. Review README completeness and accuracy.
2. Inspect .gitignore for environment, build, editor, and OS artifacts.
3. Review ten recent commits for scope and message quality.
4. Check whether the default-branch workflow requires review or automated checks.
5. Analyze a fictional exposed-token scenario and write the revoke-first response sequence.

Required evidence: A prioritized repository audit with evidence and corrected README and ignore rules

Safety and privacy: Use only fictional credential strings. If a real secret is discovered, stop sharing it, revoke it immediately, notify the instructor or owner, and follow the approved incident process.

Self-Test

1. Why avoid rewriting shared history?
2. What is the first response to an exposed token?
3. Does deleting a secret in a new commit erase history?
4. What does git check-ignore -v explain?
5. What should an audit recommendation contain?

Performance Task

Performance Task: Conduct a repository quality and secret-safety audit

1. Review README completeness and accuracy.
2. Inspect .gitignore for environment, build, editor, and OS artifacts.
3. Review ten recent commits for scope and message quality.
4. Check whether the default-branch workflow requires review or automated checks.
5. Analyze a fictional exposed-token scenario and write the revoke-first response sequence.

Required evidence: A prioritized repository audit with evidence and corrected README and ignore rules

Safety and privacy: Use only fictional credential strings. If a real secret is discovered, stop sharing it, revoke it immediately, notify the instructor or owner, and follow the approved incident process.

Post-Test

1. What should a README establish?
2. What does .gitignore affect?
3. Why ignore .env?
4. What makes a commit focused?
5. Why protect the default branch?
6. Why avoid rewriting shared history?
7. What is the first response to an exposed token?
8. Does deleting a secret in a new commit erase history?
9. What does git check-ignore -v explain?

10. What should an audit recommendation contain?

Suggested Readings

- GitHub Docs: Ignoring files
- GitHub Docs: Removing sensitive data from a repository

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Git. (2026). Git reference. <https://git-scm.com/docs>

Unit 9: GitHub Actions and Automation

Time allotment: 3 hours

Overview

Students create a least-privilege workflow that validates required portfolio files on pushes and pull requests.

Introduction and Rationale

Automated checks provide repeatable feedback before changes are merged or published.

Learning Objectives

1. Identify workflow events, jobs, runners, steps, and actions.
2. Create a valid workflow file.
3. Interpret workflow logs and results.
4. Apply least-privilege permissions and secret safety.

Learner Instructions

- Create the file under `.github/workflows`.
- Use YAML spaces consistently.
- Read the failed step before changing the workflow.

Materials

- student-portfolio repository
- GitHub Actions access
- Text editor

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. Where must workflow files be stored?
2. What starts a workflow run?
3. What is a job?
4. What is a runner?
5. What can a step do?



Figure 9. A continuous integration validation pipeline

Source: Original instructional illustration generated for this module.

Concepts and Discussion

A GitHub Actions workflow is a YAML file stored under ``.github/workflows``. Repository events such as `push` or `pull_request` can trigger one or more jobs.

A job runs on a selected runner and contains ordered steps. A step may run a shell command or invoke a reusable action. Jobs are isolated unless artifacts or outputs are passed deliberately.

The workflow should request only the permissions it needs. A validation job that reads repository files can normally use `contents: read` rather than write access.

A failed run is diagnostic evidence. Students should open the job, find the first failed step, read its command and output, reproduce the condition when possible, then make a focused correction.

Secrets belong in GitHub's encrypted secret storage and are accessed through the secrets context. Secret values must not be printed, committed, placed in screenshots, or copied into workflow source.

Command Examples

```
# .github/workflows/validate.yml
name: Validate portfolio
on: [push, pull_request]
permissions:
  contents: read
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - name: Verify required files
        shell: bash
        run: test -f README.md && test -f index.html
```

A complete least-privilege validation workflow for the portfolio repository.

```
git add .github/workflows/validate.yml
```

Stages the workflow definition.

```
git commit -m "ci: validate portfolio files"
```

Records the automation as a focused change.

```
git push
```

Publishes the workflow and triggers the push event.

Guided Learning Activity

Create a portfolio validation workflow

1. Create `.github/workflows/validate.yml` using the supplied structure.
2. Commit and push the workflow on a feature branch.
3. Inspect the run and identify its trigger, job, runner, and steps.
4. Temporarily rename one required file in the practice branch, observe the expected failure, and read the log.
5. Restore the file, push the correction, obtain a passing run, and open a pull request.

Required evidence: A workflow file, one explained failure, a corrected passing run, and a pull request check

Safety and privacy: Never place a real secret in YAML, logs, commands, commits, screenshots, or assessment evidence. Use repository secret storage only when an approved activity requires it.

Self-Test

1. Why declare permissions?
2. What should be inspected first after failure?
3. Where should approved secret values be stored?
4. What triggers the example workflow?
5. What does a passing check demonstrate?

Performance Task

Performance Task: Create a portfolio validation workflow

1. Create `.github/workflows/validate.yml` using the supplied structure.

2. Commit and push the workflow on a feature branch.
3. Inspect the run and identify its trigger, job, runner, and steps.
4. Temporarily rename one required file in the practice branch, observe the expected failure, and read the log.
5. Restore the file, push the correction, obtain a passing run, and open a pull request.

Required evidence: A workflow file, one explained failure, a corrected passing run, and a pull request check

Safety and privacy: Never place a real secret in YAML, logs, commands, commits, screenshots, or assessment evidence. Use repository secret storage only when an approved activity requires it.

Post-Test

1. Where must workflow files be stored?
2. What starts a workflow run?
3. What is a job?
4. What is a runner?
5. What can a step do?
6. Why declare permissions?
7. What should be inspected first after failure?
8. Where should approved secret values be stored?
9. What triggers the example workflow?
10. What does a passing check demonstrate?

Suggested Readings

- GitHub Docs: Quickstart for GitHub Actions
- GitHub Docs: Workflow syntax

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 10: GitHub Pages and Static-Site Publishing

Time allotment: 3 hours

Overview

Students turn the continuing portfolio repository into a public static site and verify its deployment.

Introduction and Rationale

Publishing makes repository content visible to an authentic audience and connects version control with a repeatable deployment process.

Learning Objectives

1. Explain what GitHub Pages can publish.
2. Create a valid entry page.
3. Configure a publishing source.
4. Diagnose a failed or stale deployment.

Learner Instructions

- Publish only instructor-approved content.
- Use relative links for project files.
- Verify the deployed URL in a private browser window.

Materials

- student-portfolio repository
- Web browser
- Text editor

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What type of content does GitHub Pages publish?
2. What entry file is commonly required?
3. Why prefer relative links?
4. Where is the publishing source selected?
5. Does a successful push prove deployment succeeded?

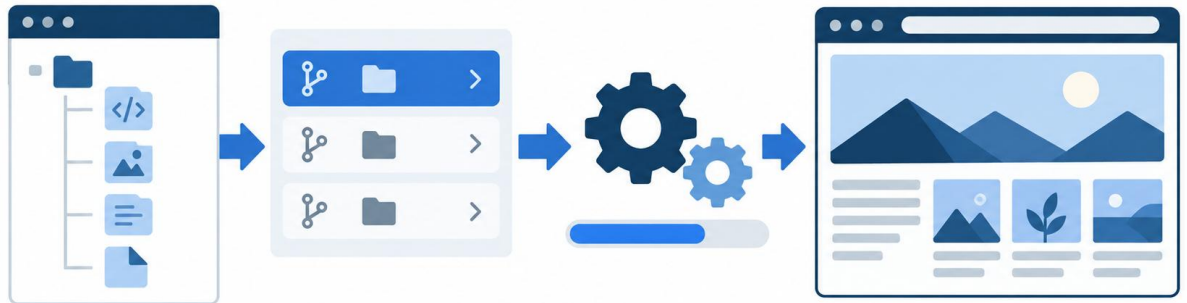


Figure 10. Publishing a static site with GitHub Pages

Source: Original instructional illustration generated for this module.

Concepts and Discussion

GitHub Pages publishes static HTML, CSS, JavaScript, and assets from a repository. It does not run a general server-side application or protect content behind course authentication.

A project site needs a clear entry point such as `index.html`. Relative paths keep styles, images, and internal links working when the site is served below an account and repository path.

Repository settings identify the publishing source. Depending on the repository, Pages can deploy from a selected branch and folder or through a GitHub Actions workflow.

A deployment has its own status and logs. A successful commit does not guarantee a successful publication; students should inspect the Pages deployment and the browser result.

Publishing requires a privacy review. A portfolio must not expose student numbers, home addresses, private contact details, grades, credentials, unpublished assessments, or copyrighted material without permission.

Command Examples

```
test -f index.html
```

Checks that the required entry page exists in a Unix-like shell.

```
git add index.html assets/
```

Stages the site entry page and intended assets.

```
git commit -m "feat: publish portfolio landing page"
```

Records the site as a focused change.

```
git push
```

Publishes the commit so the configured Pages source can deploy it.

Guided Learning Activity

Publish the student portfolio

1. Create index.html with a title, short introduction, skills list, and links to approved project work.
2. Run a privacy review and remove personal or confidential information.
3. Commit and push the static site on a reviewed branch.
4. Configure the repository's GitHub Pages source and inspect deployment status.
5. Open the public URL, test every link, and record one corrected publishing issue if present.

Required evidence: A working GitHub Pages URL, deployment status, privacy checklist, and verified navigation

Safety and privacy: Publish only instructor-approved public information; never publish credentials, private student records, home addresses, or copyrighted material without permission.

Self-Test

1. What should deployment logs be used for?
2. What content must a student portfolio exclude?
3. Why test in a private browser window?
4. What should be checked after publication?
5. What does static hosting not provide?

Performance Task

Performance Task: Publish the student portfolio

1. Create index.html with a title, short introduction, skills list, and links to approved project work.
2. Run a privacy review and remove personal or confidential information.
3. Commit and push the static site on a reviewed branch.
4. Configure the repository's GitHub Pages source and inspect deployment status.
5. Open the public URL, test every link, and record one corrected publishing issue if present.

Required evidence: A working GitHub Pages URL, deployment status, privacy checklist, and verified navigation

Safety and privacy: Publish only instructor-approved public information; never publish credentials, private student records, home addresses, or copyrighted material without permission.

Post-Test

1. What type of content does GitHub Pages publish?
2. What entry file is commonly required?
3. Why prefer relative links?
4. Where is the publishing source selected?
5. Does a successful push prove deployment succeeded?
6. What should deployment logs be used for?
7. What content must a student portfolio exclude?
8. Why test in a private browser window?
9. What should be checked after publication?

10. What does static hosting not provide?

Suggested Readings

- GitHub Docs: Quickstart for GitHub Pages
- GitHub Docs: Configuring a publishing source

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Unit 11: Security and Advanced GitHub Features

Time allotment: 2 hours

Overview

Students audit account and repository protections and survey releases, packages, insights, and API-based automation.

Introduction and Rationale

Security and advanced features are most useful when students understand access boundaries, alerts, and the limits of each capability.

Learning Objectives

1. Apply account and repository security controls.
2. Interpret dependency and code-security findings.
3. Explain releases, packages, insights, and API use cases.
4. Prioritize repository improvements using risk and effort.

Learner Instructions

- Use a sample repository for the audit.
- Do not enable paid features for this activity.
- Separate observed evidence from recommendations.

Materials

- GitHub account
- Sample repository
- Security audit checklist

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. Why enable two-factor authentication?
2. What is least privilege?
3. What should happen after a real secret is exposed?
4. What do Dependabot alerts report?
5. What does dependency review show?



Figure 11. Layered protection for an account and repository

Source: Original instructional illustration generated for this module.

Concepts and Discussion

Account security begins with a unique password, two-factor authentication, protected recovery methods, and careful review of authorized applications and active sessions.

Least privilege means granting each collaborator, workflow, token, and integration only the access required for its task. Access should be reviewed when roles or projects change.

Secret scanning can identify supported credential patterns, while push protection may block known secret formats before they enter history. Detection does not replace immediate revocation of an exposed credential.

Dependabot alerts identify known vulnerabilities in supported dependencies, and dependency review explains changes introduced by a pull request. Code scanning analyzes code for configured classes of security weakness.

Tags and releases mark meaningful versions; Packages distributes supported artifacts; Insights summarizes repository activity; and the GitHub API enables authorized automation. These features should solve a defined need rather than be enabled for decoration.

Command Examples

```
git tag -a v1.0.0 -m "Release portfolio version 1.0.0"
```

Creates an annotated local release tag.

```
git push origin v1.0.0
```

Publishes the approved tag.

```
gh api repos/OWNER/REPOSITORY
```

Optional authenticated API request for repository metadata; replace placeholders and never expose a token.

Guided Learning Activity

Audit repository security and advanced-feature readiness

1. Confirm two-factor authentication is enabled without capturing recovery codes.
2. Review collaborators and workflow permissions for least privilege.
3. Inspect available dependency, secret, and code-scanning findings.
4. Evaluate whether a release, package, insight, or API use case serves the portfolio project.
5. Recommend three improvements ranked by risk reduction, learning value, and implementation effort.

Required evidence: A redacted security audit with three prioritized and justified improvements

Safety and privacy: Do not include passwords, tokens, private keys, recovery codes, private alerts, or sensitive repository content in submitted evidence.

Self-Test

1. What does code scanning analyze?
2. What is an annotated tag used for?
3. What is GitHub Packages for?
4. What can Insights provide?
5. When should the GitHub API be used?

Performance Task

Performance Task: Audit repository security and advanced-feature readiness

1. Confirm two-factor authentication is enabled without capturing recovery codes.
2. Review collaborators and workflow permissions for least privilege.
3. Inspect available dependency, secret, and code-scanning findings.
4. Evaluate whether a release, package, insight, or API use case serves the portfolio project.
5. Recommend three improvements ranked by risk reduction, learning value, and implementation effort.

Required evidence: A redacted security audit with three prioritized and justified improvements

Safety and privacy: Do not include passwords, tokens, private keys, recovery codes, private alerts, or sensitive repository content in submitted evidence.

Post-Test

1. Why enable two-factor authentication?
2. What is least privilege?
3. What should happen after a real secret is exposed?
4. What do Dependabot alerts report?
5. What does dependency review show?
6. What does code scanning analyze?
7. What is an annotated tag used for?
8. What is GitHub Packages for?
9. What can Insights provide?
10. When should the GitHub API be used?

Suggested Readings

- GitHub Docs: Securing your account
- GitHub Docs: Securing your repository

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Git. (2026). Git reference. <https://git-scm.com/docs>

Unit 12: Contributing to Open Source Projects

Time allotment: 2 hours

Overview

Students prepare a respectful, policy-compliant contribution using project documentation, a focused branch, and review feedback.

Introduction and Rationale

Open-source contribution combines technical Git skills with licensing awareness, community norms, and clear communication.

Learning Objectives

1. Evaluate whether a project welcomes a proposed contribution.
2. Follow repository community and contribution files.
3. Prepare a focused fork-based change.
4. Respond constructively to maintainer review.

Learner Instructions

- Use only an instructor-approved project.
- Read community files before editing.
- Do not claim an issue without following project norms.

Materials

- Instructor-approved repository
- GitHub account
- Contribution checklist

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What makes a project suitable for a first contribution?
2. Which repository files should be read first?
3. What is a fork?
4. What is the upstream remote?
5. Why synchronize before branching?

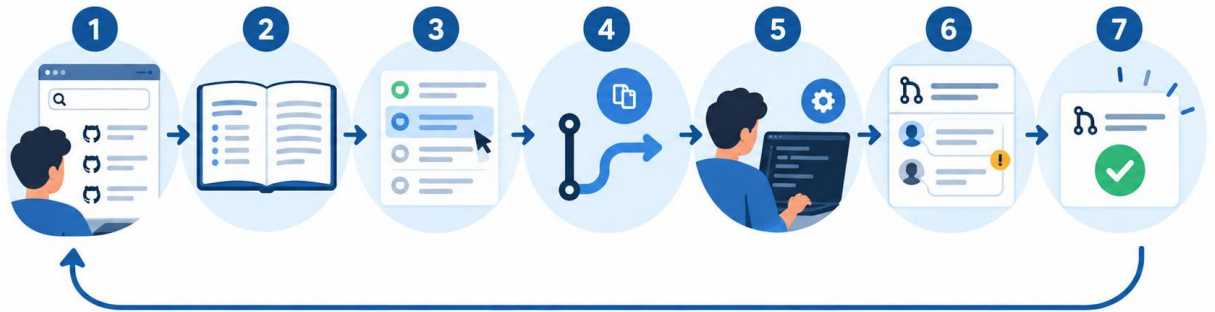


Figure 12. An open source contribution journey

Source: Original instructional illustration generated for this module.

Concepts and Discussion

A suitable first contribution has clear scope, active maintainers, an understandable contribution guide, and a license. Documentation corrections and small tests often provide a safer entry than a large feature.

README, CONTRIBUTING, CODE_OF_CONDUCT, issue templates, pull-request templates, and license files define how a community expects work to proceed. Repository-specific rules override generic advice.

A fork creates a contributor-owned remote copy. The contributor clones the fork, adds the original repository as `upstream`, synchronizes from upstream, and creates a focused branch for the proposed change.

A good contribution explains motivation, limits its diff, includes validation evidence, and avoids unrelated cleanup. Licensing and attribution requirements must be respected for any reused material.

Maintainer feedback is part of the contribution. Contributors should ask clarifying questions, update the same pull request with focused commits, and accept that maintainers may decline work that does not fit the project.

Command Examples

```
git remote add upstream https://github.com/ORIGINAL/PROJECT.git
```

Connects the local clone to the original repository.

```
git fetch upstream
```

Downloads the original repository's current history.

```
git switch -c docs/clarify-setup upstream/main
```

Creates a focused contribution branch from current upstream main.

```
git push -u origin docs/clarify-setup
```

Publishes the branch to the contributor's fork.

Guided Learning Activity

Prepare an open-source contribution

1. Select an instructor-approved project and read its README, contribution guide, code of conduct, and license.
2. Identify one small issue or documentation improvement and confirm that it is not already being addressed.
3. Fork, clone, add upstream, synchronize, and create a focused branch.
4. Make the smallest complete change and validate it according to project instructions.
5. Prepare a pull-request description or submit the contribution when the instructor authorizes external interaction.

Required evidence: A contribution plan or documentation pull request that follows the project's stated policies

Safety and privacy: Do not submit to an external project without instructor approval, and never include credentials, private course work, or copyrighted content without permission.

Self-Test

1. What makes a contribution focused?
2. Why check the license?
3. How should review feedback be handled?
4. Can a maintainer decline a technically correct change?
5. What requires instructor approval in this unit?

Performance Task

Performance Task: Prepare an open-source contribution

1. Select an instructor-approved project and read its README, contribution guide, code of conduct, and license.
2. Identify one small issue or documentation improvement and confirm that it is not already being addressed.
3. Fork, clone, add upstream, synchronize, and create a focused branch.
4. Make the smallest complete change and validate it according to project instructions.
5. Prepare a pull-request description or submit the contribution when the instructor authorizes external interaction.

Required evidence: A contribution plan or documentation pull request that follows the project's stated policies

Safety and privacy: Do not submit to an external project without instructor approval, and never include credentials, private course work, or copyrighted content without permission.

Post-Test

1. What makes a project suitable for a first contribution?
2. Which repository files should be read first?
3. What is a fork?
4. What is the upstream remote?
5. Why synchronize before branching?

6. What makes a contribution focused?
7. Why check the license?
8. How should review feedback be handled?
9. Can a maintainer decline a technically correct change?
10. What requires instructor approval in this unit?

Suggested Readings

- GitHub Docs: Contributing to open source
- Open Source Guides: How to Contribute

References

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

GitHub. (2026). Open Source Guides. <https://opensource.guide/>

Unit 13: Troubleshooting and Recovery

Time allotment: 3 hours

Overview

Students diagnose repository state before choosing a safe repair and document what evidence justified the action.

Introduction and Rationale

Recovery becomes safer when students observe first, preserve work, choose the least destructive action, and verify the result.

Learning Objectives

1. Collect diagnostic evidence before repair.
2. Resolve common synchronization and authentication failures.
3. Recover staged, working-tree, and committed work safely.
4. Explain when destructive history operations require escalation.

Learner Instructions

- Stop after an unexpected result.
- Copy important uncommitted files before risky recovery.
- Use the least destructive command that solves the diagnosed problem.

Materials

- Instructor-provided broken repositories
- Terminal
- Recovery decision sheet

Pre-Test

Answer each item before studying the unit. Check your work only after completing the unit.

1. What evidence should be collected first?
2. Why fetch before repairing a rejected push?
3. What should happen if a credential was exposed?
4. How can staged content be unstaged safely?
5. When is git restore FILE dangerous?

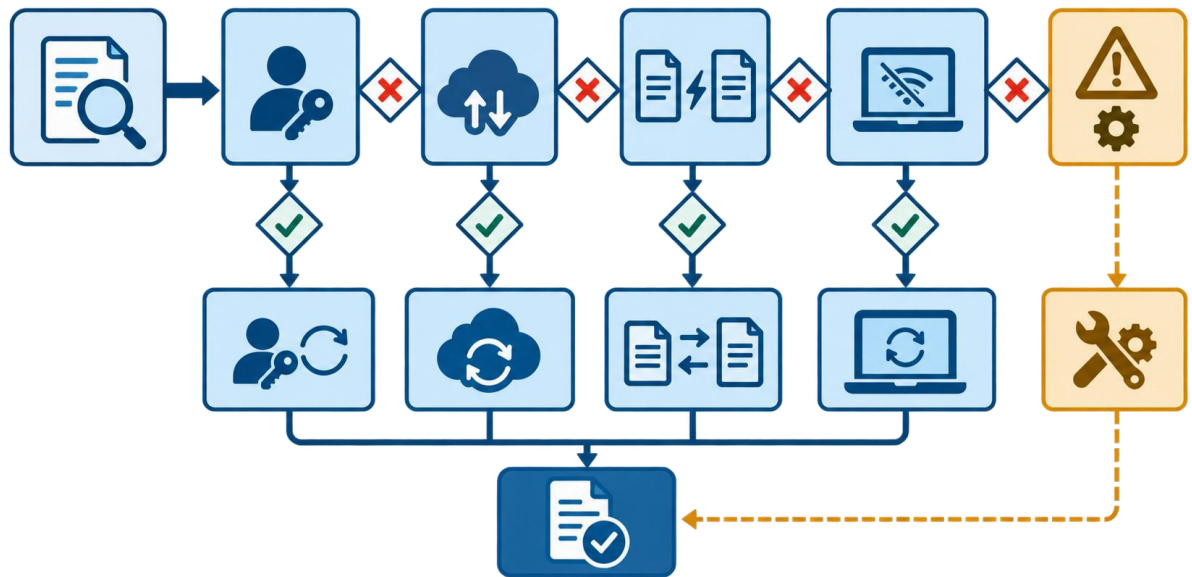


Figure 13. A safe first troubleshooting decision path

Source: Original instructional illustration generated for this module.

Concepts and Discussion

Troubleshooting begins with the exact command, complete error, current branch, status, recent log, remote URLs, and any action immediately before failure. Guessing with random commands can destroy the evidence needed for recovery.

A rejected push often means remote history has commits not present locally or branch rules require a pull request. Fetch, inspect the graph, and choose an approved integration path rather than forcing the push.

Authentication failures require checking the remote protocol, credential helper or SSH agent, account access, token scope, and repository permission. If a credential is exposed, revoke it before continuing.

``git restore`` can discard an unwanted working-tree edit or unstage content depending on its options. ``git revert`` records a new commit that reverses an earlier commit and is usually safer for shared history than rewriting it.

``git reflog`` records recent local reference movement and can help locate an apparently lost commit. A detached HEAD can be preserved by creating a branch at the valuable commit before switching away.

``git reset --hard`` and force-push can permanently discard reachable work or replace shared history. They belong only in an instructor-led recovery after a backup, diagnosis, communication, and confirmation that safer alternatives are insufficient.

Command Examples

```
git status
```

Captures branch and file state before repair.

```
git log --oneline --graph --decorate --all -15
```

Shows recent local and remote-tracking history.

```
git remote -v
```

Confirms the actual fetch and push destinations.

```
git reflog -15
```

Locates recent local branch and HEAD movement.

```
git restore --staged FILE
```

Unstages a file while preserving its working-tree content.

```
git restore FILE
```

Discards an unwanted working-tree change after verification and backup.

```
git revert COMMIT
```

Creates a new commit that reverses a shared change.

```
git switch -c recovered-work COMMIT
```

Preserves a valuable commit by creating a branch.

```
git reset --hard COMMIT
```

Instructor-led last resort that resets index and working tree and can discard work.

Guided Learning Activity

Diagnose and recover four controlled failures

1. For each broken practice repository, record the command, error, status, graph, and remote configuration.
2. Classify the problem as authentication, synchronization, conflict, detached HEAD, staging, or recovery.
3. Choose the least destructive repair and predict its effect on working tree, index, history, and remote.
4. Apply the repair, verify status and history, and record evidence.
5. Compare the safe solution with an instructor-demonstrated destructive alternative and explain why escalation is required.

Required evidence: A four-case troubleshooting log showing evidence, diagnosis, selected repair, predicted effect, and verified result

Safety and privacy: Never use `reset --hard`, force push, or another history rewrite outside an instructor-led practice repository. Make a backup first and confirm that the command cannot affect shared work.

Self-Test

1. Why is `git revert` suitable for shared history?
2. What does `git reflog help locate`?
3. How can detached work be preserved?
4. Why is force-push risky?
5. When is `reset --hard` acceptable in this module?

Performance Task

Performance Task: Diagnose and recover four controlled failures

1. For each broken practice repository, record the command, error, status, graph, and remote configuration.
2. Classify the problem as authentication, synchronization, conflict, detached HEAD, staging, or recovery.
3. Choose the least destructive repair and predict its effect on working tree, index, history, and remote.

4. Apply the repair, verify status and history, and record evidence.
5. Compare the safe solution with an instructor-demonstrated destructive alternative and explain why escalation is required.

Required evidence: A four-case troubleshooting log showing evidence, diagnosis, selected repair, predicted effect, and verified result

Safety and privacy: Never use `reset --hard`, force push, or another history rewrite outside an instructor-led practice repository. Make a backup first and confirm that the command cannot affect shared work.

Post-Test

1. What evidence should be collected first?
2. Why fetch before repairing a rejected push?
3. What should happen if a credential was exposed?
4. How can staged content be unstaged safely?
5. When is `git restore FILE` dangerous?
6. Why is `git revert` suitable for shared history?
7. What does `git reflog` help locate?
8. How can detached work be preserved?
9. Why is force-push risky?
10. When is `reset --hard` acceptable in this module?

Suggested Readings

- Git reference: `git-restore`
- Git reference: `git-reflog`
- GitHub Docs: Troubleshooting connectivity problems

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

Capstone Project

Capstone: Publish a Reviewed Student Portfolio

1. Verify Git identity and repository authentication without submitting any credential.
2. Create or refine the student-portfolio repository with an accurate README and safe .gitignore.
3. Create an issue with acceptance criteria and track it on a project view.
4. Implement the issue on a focused feature branch with small, descriptive commits.
5. Open a pull request, obtain peer review, address feedback, and merge after checks pass.
6. Add a least-privilege validation workflow under .github/workflows.
7. Publish the approved static portfolio through GitHub Pages and verify the public site.
8. Submit a reflection connecting repository evidence to the Git and GitHub concepts learned.

Required evidence: Repository URL, issue, project evidence, feature branch history, reviewed pull request, passing workflow, GitHub Pages URL, and reflection

Safety and privacy: Exclude credentials, private keys, tokens, private student data, grades, and unlicensed material from the repository and evidence.

Assessment Rubrics

Unit Performance Task Rubric

Criterion	Performance descriptors
Correctness	4: accurate and complete; 3: minor correction; 2: partial; 1: incorrect or missing
Process	4: deliberate safe workflow; 3: one avoidable step; 2: repeated guidance needed; 1: unsafe or undocumented
Evidence	4: sufficient and clearly labeled; 3: mostly sufficient; 2: incomplete; 1: absent
Safe practice	4: protects credentials and data; 3: minor privacy correction; 2: unsafe tendency corrected; 1: exposed or mishandled sensitive data

Pull Request Rubric

Criterion	Performance descriptors
-----------	-------------------------

Criterion	Performance descriptors
Scope	4: one focused issue; 3: small unrelated change; 2: broad diff; 1: purpose unclear
Description	4: rationale, changes, evidence, and issue link; 3: one item weak; 2: several missing; 1: absent
Review response	4: respectful and evidence-based; 3: addressed with minor gaps; 2: incomplete; 1: ignored or inappropriate
Checks	4: all required checks pass; 3: nonblocking warning explained; 2: failure unresolved; 1: no validation

Capstone Rubric

Criterion	Performance descriptors
Repository correctness	4: complete and functional; 3: minor defect; 2: multiple gaps; 1: unusable
Git history	4: focused, readable commits; 3: mostly focused; 2: inconsistent; 1: missing or confusing
Collaboration	4: traceable issue, review, revision, merge; 3: one gap; 2: limited evidence; 1: absent
Documentation	4: accurate README and reflection; 3: minor omissions; 2: incomplete; 1: absent
Automation	4: useful least-privilege passing workflow; 3: works with minor issue; 2: unreliable; 1: absent
Publication	4: accessible verified Pages site; 3: minor defect; 2: unstable; 1: absent
Security	4: safe data and access practice; 3: minor correction; 2: material concern corrected; 1: exposed sensitive data
Reflection	4: evidence-based synthesis; 3: sound but general; 2: superficial; 1: absent

Answer Key

Instructor copy. Accept equivalent wording when it demonstrates the same accurate concept.

Unit 1: Introduction to Git and GitHub

Item	Expected answer
u1-pre-1	It records ordered project states so changes can be explained, compared, shared, and recovered.
u1-pre-2	Git is the local distributed version control system; GitHub is a hosted collaboration platform built around Git repositories.
u1-pre-3	A commit records staged content, metadata, and links to its parent history.
u1-pre-4	It is the index holding the exact content proposed for the next commit.
u1-pre-5	A branch is a movable name that points to a line of commits.
u1-self-1	It creates a local repository with project files, history, and a configured remote.
u1-self-2	A remote supports sharing, backup, collaboration, review, and automation.
u1-self-3	It is the checked-out set of files currently available for editing.
u1-self-4	Fetch downloads remote history without integrating it; pull fetches and then integrates.
u1-self-5	They explain the purpose of a recorded change to reviewers and future maintainers.
u1-post-1	It records ordered project states so changes can be explained, compared, shared, and recovered.
u1-post-2	Git is the local distributed version control system; GitHub is a hosted collaboration platform built around Git repositories.

Item	Expected answer
u1-post-3	A commit records staged content, metadata, and links to its parent history.
u1-post-4	It is the index holding the exact content proposed for the next commit.
u1-post-5	A branch is a movable name that points to a line of commits.
u1-post-6	It creates a local repository with project files, history, and a configured remote.
u1-post-7	A remote supports sharing, backup, collaboration, review, and automation.
u1-post-8	It is the checked-out set of files currently available for editing.
u1-post-9	Fetch downloads remote history without integrating it; pull fetches and then integrates.
u1-post-10	They explain the purpose of a recorded change to reviewers and future maintainers.

Unit 2: Setting Up Git and GitHub

Item	Expected answer
u2-pre-1	Run <code>git --version</code> and confirm that it reports an installed version.
u2-pre-2	It supplies the author name recorded in new commits.
u2-pre-3	It supplies the author email recorded in new commits and can link commits to a verified GitHub identity.
u2-pre-4	No. It is commit metadata, not a credential.
u2-pre-5	Global values apply to the current user by default; local values apply only to one repository and override global values.

Item	Expected answer
u2-self-1	Only the public key; the private key must remain protected on the device.
u2-self-2	It uses a credential manager or token-based credential rather than an account password.
u2-self-3	Tokens, passwords, private keys, recovery codes, and unrelated personal data.
u2-self-4	Because correcting authorship after commits are shared may require history rewriting.
u2-self-5	It proves the credential identifies the account; repository permissions still determine allowed actions.
u2-post-1	Run <code>git --version</code> and confirm that it reports an installed version.
u2-post-2	It supplies the author name recorded in new commits.
u2-post-3	It supplies the author email recorded in new commits and can link commits to a verified GitHub identity.
u2-post-4	No. It is commit metadata, not a credential.
u2-post-5	Global values apply to the current user by default; local values apply only to one repository and override global values.
u2-post-6	Only the public key; the private key must remain protected on the device.
u2-post-7	It uses a credential manager or token-based credential rather than an account password.
u2-post-8	Tokens, passwords, private keys, recovery codes, and unrelated personal data.

Item	Expected answer
u2-post-9	Because correcting authorship after commits are shared may require history rewriting.
u2-post-10	It proves the credential identifies the account; repository permissions still determine allowed actions.

Unit 3: Creating the First Repository

Item	Expected answer
u3-pre-1	A local repository with project files, history, a checked-out branch, and an origin remote.
u3-pre-2	It creates a local Git repository in the current directory; it does not create or upload a GitHub repository.
u3-pre-3	It confirms the repository's fetch and push destinations before synchronization.
u3-pre-4	A clear purpose, scope, and basic project information without private data.
u3-pre-5	It confirms that only intended changes are staged and reveals untracked or unstaged files.
u3-self-1	It is the conventional name for the primary remote, commonly created by cloning.
u3-self-2	It records the upstream relationship between the local and remote branch.
u3-self-3	Authentication, authorization, branch rules, an incorrect remote, or remote commits not present locally can cause rejection.
u3-self-4	Create the remote with a README and clone it, because the remote and default branch are configured automatically.

Item	Expected answer
u3-self-5	Secrets and unnecessary personal information.
u3-post-1	A local repository with project files, history, a checked-out branch, and an origin remote.
u3-post-2	It creates a local Git repository in the current directory; it does not create or upload a GitHub repository.
u3-post-3	It confirms the repository's fetch and push destinations before synchronization.
u3-post-4	A clear purpose, scope, and basic project information without private data.
u3-post-5	It confirms that only intended changes are staged and reveals untracked or unstaged files.
u3-post-6	It is the conventional name for the primary remote, commonly created by cloning.
u3-post-7	It records the upstream relationship between the local and remote branch.
u3-post-8	Authentication, authorization, branch rules, an incorrect remote, or remote commits not present locally can cause rejection.
u3-post-9	Create the remote with a README and clone it, because the remote and default branch are configured automatically.
u3-post-10	Secrets and unnecessary personal information.

Unit 4: Essential Git Commands

Item	Expected answer
u4-pre-1	It reports branch and upstream state plus staged, unstaged, and untracked changes.

Item	Expected answer
u4-pre-2	It shows unstaged changes between the working tree and staging area.
u4-pre-3	It shows the content staged for the next commit compared with the current commit.
u4-pre-4	It copies selected current file content into the staging area.
u4-pre-5	It records staged content as a new local commit with metadata and a parent relationship.
u4-self-1	They make changes easier to review, explain, test, revert, and combine.
u4-self-2	It updates remote-tracking data without integrating changes into the current branch.
u4-self-3	Pull fetches and then integrates; fetch only downloads remote history.
u4-self-4	After local commits are ready to publish to an authorized remote branch.
u4-self-5	Tracked files match the current commit and no staged or unstaged changes remain.
u4-post-1	It reports branch and upstream state plus staged, unstaged, and untracked changes.
u4-post-2	It shows unstaged changes between the working tree and staging area.
u4-post-3	It shows the content staged for the next commit compared with the current commit.
u4-post-4	It copies selected current file content into the staging area.
u4-post-5	It records staged content as a new local commit with metadata and a parent relationship.

Item	Expected answer
u4-post-6	They make changes easier to review, explain, test, revert, and combine.
u4-post-7	It updates remote-tracking data without integrating changes into the current branch.
u4-post-8	Pull fetches and then integrates; fetch only downloads remote history.
u4-post-9	After local commits are ready to publish to an authorized remote branch.
u4-post-10	Tracked files match the current commit and no staged or unstaged changes remain.

Unit 5: Branching and Merging

Item	Expected answer
u5-pre-1	A branch points to the latest commit in its line of development.
u5-pre-2	It creates a new branch from the current commit and checks it out.
u5-pre-3	It prevents committing work to the wrong line of development.
u5-pre-4	When the target has no new divergent commits and can move directly to the source tip.
u5-pre-5	Competing changes that Git cannot safely combine automatically.
u5-self-1	They delimit the current and incoming versions that require a human resolution.
u5-self-2	To produce the correct final file, not merely remove marker lines.
u5-self-3	Review it, stage it, and complete the merge commit when required.

Item	Expected answer
u5-self-4	After its work is integrated and verified or otherwise preserved.
u5-self-5	It confirms branch relationships, divergence, and merge results.
u5-post-1	A branch points to the latest commit in its line of development.
u5-post-2	It creates a new branch from the current commit and checks it out.
u5-post-3	It prevents committing work to the wrong line of development.
u5-post-4	When the target has no new divergent commits and can move directly to the source tip.
u5-post-5	Competing changes that Git cannot safely combine automatically.
u5-post-6	They delimit the current and incoming versions that require a human resolution.
u5-post-7	To produce the correct final file, not merely remove marker lines.
u5-post-8	Review it, stage it, and complete the merge commit when required.
u5-post-9	After its work is integrated and verified or otherwise preserved.
u5-post-10	It confirms branch relationships, divergence, and merge results.

Unit 6: Collaborating with Teams

Item	Expected answer
u6-pre-1	When contributors have authorized write access to the same repository.

Item	Expected answer
u6-pre-2	When contributors do not have write access or the project requires fork-based contributions.
u6-pre-3	Its purpose, scope, related issue, important changes, and validation evidence.
u6-pre-4	The combined change proposed from the source branch to the target branch.
u6-pre-5	A specific observation, its impact, and a clear requested outcome or question.
u6-self-1	Push additional commits to its source branch.
u6-self-2	It connects implementation to the requirement and can automate closure when merged.
u6-self-3	Required reviews, checks, conflict resolution, and repository rules must be satisfied.
u6-self-4	It combines the branch's changes into one commit on the target branch.
u6-self-5	It documents decisions, feedback, revisions, and approval.
u6-post-1	When contributors have authorized write access to the same repository.
u6-post-2	When contributors do not have write access or the project requires fork-based contributions.
u6-post-3	Its purpose, scope, related issue, important changes, and validation evidence.
u6-post-4	The combined change proposed from the source branch to the target branch.
u6-post-5	A specific observation, its impact, and a clear requested outcome or question.

Item	Expected answer
u6-post-6	Push additional commits to its source branch.
u6-post-7	It connects implementation to the requirement and can automate closure when merged.
u6-post-8	Required reviews, checks, conflict resolution, and repository rules must be satisfied.
u6-post-9	It combines the branch's changes into one commit on the target branch.
u6-post-10	It documents decisions, feedback, revisions, and approval.

Unit 7: Managing Issues and Projects

Item	Expected answer
u7-pre-1	A clear outcome, sufficient context, and verifiable acceptance criteria.
u7-pre-2	The specific problem or outcome in concise language.
u7-pre-3	They classify work using a shared vocabulary.
u7-pre-4	The person responsible for the next action or completion.
u7-pre-5	Related issues and pull requests toward a target outcome or period.
u7-self-1	To visualize and update work state across issues and pull requests.
u7-self-2	Accurate status lets the team coordinate and identify blocked work.
u7-self-3	Use a supported closing keyword with the issue reference and merge the pull request.

Item	Expected answer
u7-self-4	Links or results that demonstrate each acceptance criterion was met.
u7-self-5	Credentials, private student data, grades, and confidential feedback.
u7-post-1	A clear outcome, sufficient context, and verifiable acceptance criteria.
u7-post-2	The specific problem or outcome in concise language.
u7-post-3	They classify work using a shared vocabulary.
u7-post-4	The person responsible for the next action or completion.
u7-post-5	Related issues and pull requests toward a target outcome or period.
u7-post-6	To visualize and update work state across issues and pull requests.
u7-post-7	Accurate status lets the team coordinate and identify blocked work.
u7-post-8	Use a supported closing keyword with the issue reference and merge the pull request.
u7-post-9	Links or results that demonstrate each acceptance criterion was met.
u7-post-10	Credentials, private student data, grades, and confidential feedback.

Unit 8: Repository and Workflow Best Practices

Item	Expected answer
u8-pre-1	Project purpose, use or viewing instructions, current scope, and contribution guidance when relevant.

Item	Expected answer
u8-pre-2	It affects untracked paths that Git should ignore; it does not remove already tracked files.
u8-pre-3	It commonly contains environment-specific configuration and secrets that should not enter version history.
u8-pre-4	Its changes serve one coherent purpose that can be explained and reviewed together.
u8-pre-5	To require review and validation before shared production history changes.
u8-self-1	Other collaborators may have based work on commit identifiers that rewriting replaces.
u8-self-2	Revoke or rotate the credential so the exposed value no longer grants access.
u8-self-3	No. Earlier commits still contain the value.
u8-self-4	The matching ignore rule and file responsible for it.
u8-self-5	A specific finding, evidence, impact, and prioritized corrective action.
u8-post-1	Project purpose, use or viewing instructions, current scope, and contribution guidance when relevant.
u8-post-2	It affects untracked paths that Git should ignore; it does not remove already tracked files.
u8-post-3	It commonly contains environment-specific configuration and secrets that should not enter version history.
u8-post-4	Its changes serve one coherent purpose that can be explained and reviewed together.

Item	Expected answer
u8-post-5	To require review and validation before shared production history changes.
u8-post-6	Other collaborators may have based work on commit identifiers that rewriting replaces.
u8-post-7	Revoke or rotate the credential so the exposed value no longer grants access.
u8-post-8	No. Earlier commits still contain the value.
u8-post-9	The matching ignore rule and file responsible for it.
u8-post-10	A specific finding, evidence, impact, and prioritized corrective action.

Unit 9: GitHub Actions and Automation

Item	Expected answer
u9-pre-1	In the repository's <code>.github/workflows</code> directory with a <code>.yml</code> or <code>.yaml</code> extension.
u9-pre-2	A configured event, schedule, manual dispatch, or other supported trigger.
u9-pre-3	A set of steps executed on the same runner.
u9-pre-4	The machine environment that executes a workflow job.
u9-pre-5	Run a shell command or use a reusable action.
u9-self-1	To limit the workflow token to the access the job actually requires.
u9-self-2	The first failed step and its command output.
u9-self-3	In GitHub's encrypted secrets store, not in the workflow file.

Item	Expected answer
u9-self-4	Push and pull_request events.
u9-self-5	That the defined automated conditions succeeded for that commit; it does not prove every possible defect is absent.
u9-post-1	In the repository's .github/workflows directory with a .yml or .yaml extension.
u9-post-2	A configured event, schedule, manual dispatch, or other supported trigger.
u9-post-3	A set of steps executed on the same runner.
u9-post-4	The machine environment that executes a workflow job.
u9-post-5	Run a shell command or use a reusable action.
u9-post-6	To limit the workflow token to the access the job actually requires.
u9-post-7	The first failed step and its command output.
u9-post-8	In GitHub's encrypted secrets store, not in the workflow file.
u9-post-9	Push and pull_request events.
u9-post-10	That the defined automated conditions succeeded for that commit; it does not prove every possible defect is absent.

Unit 10: GitHub Pages and Static-Site Publishing

Item	Expected answer
u10-pre-1	Static web content such as HTML, CSS, JavaScript, and assets.
u10-pre-2	An index.html file at the configured publishing source.

Item	Expected answer
u10-pre-3	They continue to work when the project site is served beneath a repository-specific path.
u10-pre-4	In the repository's Pages settings or through a configured Pages workflow.
u10-pre-5	No. The Pages deployment status and public URL must also be checked.
u10-self-1	To identify the failing build or publication step and its error message.
u10-self-2	Credentials, private records, home addresses, grades, and material lacking publication permission.
u10-self-3	It helps verify that the public site works without relying on a signed-in session.
u10-self-4	The URL, page content, navigation, assets, privacy, and deployment status.
u10-self-5	A general server-side runtime or private course access control.
u10-post-1	Static web content such as HTML, CSS, JavaScript, and assets.
u10-post-2	An index.html file at the configured publishing source.
u10-post-3	They continue to work when the project site is served beneath a repository-specific path.
u10-post-4	In the repository's Pages settings or through a configured Pages workflow.
u10-post-5	No. The Pages deployment status and public URL must also be checked.
u10-post-6	To identify the failing build or publication step and its error message.
u10-post-7	Credentials, private records, home addresses, grades, and material lacking publication permission.

Item	Expected answer
u10-post-8	It helps verify that the public site works without relying on a signed-in session.
u10-post-9	The URL, page content, navigation, assets, privacy, and deployment status.
u10-post-10	A general server-side runtime or private course access control.

Unit 11: Security and Advanced GitHub Features

Item	Expected answer
u11-pre-1	It requires a second factor and reduces risk from a stolen password.
u11-pre-2	Granting only the access needed for a specific task and duration.
u11-pre-3	Revoke or rotate it immediately, then follow the incident and cleanup process.
u11-pre-4	Known vulnerabilities affecting supported dependencies in the repository.
u11-pre-5	Dependency changes and risk information introduced by a pull request.
u11-self-1	Source code for configured patterns of security weakness.
u11-self-2	To give a durable name and metadata to a specific commit, often for a release.
u11-self-3	Publishing and consuming supported software packages or container artifacts.
u11-self-4	Views of repository activity, contributors, traffic, dependencies, and related metrics when available.
u11-self-5	When an authorized, repeatable integration or automation has a defined need.

Item	Expected answer
u11-post-1	It requires a second factor and reduces risk from a stolen password.
u11-post-2	Granting only the access needed for a specific task and duration.
u11-post-3	Revoke or rotate it immediately, then follow the incident and cleanup process.
u11-post-4	Known vulnerabilities affecting supported dependencies in the repository.
u11-post-5	Dependency changes and risk information introduced by a pull request.
u11-post-6	Source code for configured patterns of security weakness.
u11-post-7	To give a durable name and metadata to a specific commit, often for a release.
u11-post-8	Publishing and consuming supported software packages or container artifacts.
u11-post-9	Views of repository activity, contributors, traffic, dependencies, and related metrics when available.
u11-post-10	When an authorized, repeatable integration or automation has a defined need.

Unit 12: Contributing to Open Source Projects

Item	Expected answer
u12-pre-1	Clear scope, active maintenance, understandable contribution guidance, and a license.
u12-pre-2	README, contribution guide, code of conduct, templates, and license.
u12-pre-3	A GitHub-hosted copy under another account for independent contribution work.

Item	Expected answer
u12-pre-4	A local remote name conventionally pointing to the original project repository.
u12-pre-5	To base the contribution on current project history and reduce conflicts.
u12-self-1	It solves one agreed problem without unrelated changes.
u12-self-2	It defines permissions and obligations for using and contributing material.
u12-self-3	Clarify respectfully, make focused revisions, and update the same pull request.
u12-self-4	Yes. It may not fit project scope, timing, design, or maintenance priorities.
u12-self-5	Any actual interaction or submission to an external project.
u12-post-1	Clear scope, active maintenance, understandable contribution guidance, and a license.
u12-post-2	README, contribution guide, code of conduct, templates, and license.
u12-post-3	A GitHub-hosted copy under another account for independent contribution work.
u12-post-4	A local remote name conventionally pointing to the original project repository.
u12-post-5	To base the contribution on current project history and reduce conflicts.
u12-post-6	It solves one agreed problem without unrelated changes.
u12-post-7	It defines permissions and obligations for using and contributing material.
u12-post-8	Clarify respectfully, make focused revisions, and update the same pull request.

Item	Expected answer
u12-post-9	Yes. It may not fit project scope, timing, design, or maintenance priorities.
u12-post-10	Any actual interaction or submission to an external project.

Unit 13: Troubleshooting and Recovery

Item	Expected answer
u13-pre-1	The exact command and error, status, branch, recent graph, remote URLs, and preceding action.
u13-pre-2	It reveals remote history without immediately changing the current branch.
u13-pre-3	Revoke or rotate it immediately before repository cleanup.
u13-pre-4	Use <code>git restore --staged FILE</code> while keeping the working-tree content.
u13-pre-5	When the current working-tree changes are valuable and not backed up, because the command discards them.
u13-self-1	It preserves existing commits and records a new inverse change.
u13-self-2	Recent local reference and HEAD positions, including commits no longer named by a branch.
u13-self-3	Create a new branch pointing to the valuable commit before switching away.
u13-self-4	It can replace remote history that collaborators depend on.
u13-self-5	Only as an instructor-led last resort in a backed-up practice repository after safer options are rejected.
u13-post-1	The exact command and error, status, branch, recent graph, remote URLs, and preceding action.

Item	Expected answer
u13-post-2	It reveals remote history without immediately changing the current branch.
u13-post-3	Revoke or rotate it immediately before repository cleanup.
u13-post-4	Use <code>git restore --staged FILE</code> while keeping the working-tree content.
u13-post-5	When the current working-tree changes are valuable and not backed up, because the command discards them.
u13-post-6	It preserves existing commits and records a new inverse change.
u13-post-7	Recent local reference and HEAD positions, including commits no longer named by a branch.
u13-post-8	Create a new branch pointing to the valuable commit before switching away.
u13-post-9	It can replace remote history that collaborators depend on.
u13-post-10	Only as an instructor-led last resort in a backed-up practice repository after safer options are rejected.

Git Command Cheat Sheet

Command	Purpose
<code>git --version</code>	Verify the Git installation.
<code>git config --global user.name "Name"</code>	Set the default commit author name.
<code>git config --global user.email "email"</code>	Set the default commit author email.
<code>git init</code>	Create a repository in the current folder.
<code>git clone <url></code>	Create a local clone of a remote repository.

Command	Purpose
git status	Inspect branch, staging, and working-tree state.
git diff	Review unstaged changes.
git diff --staged	Review staged changes.
git add <path>	Stage a selected path.
git commit -m "message"	Record staged content.
git log --oneline --graph	Inspect compact repository history.
git branch	List local branches.
git switch -c <branch>	Create and switch to a feature branch.
git switch <branch>	Switch to an existing branch.
git merge <branch>	Integrate a branch into the current branch.
git remote -v	Inspect configured remotes.
git fetch	Download remote history without integrating it.
git pull	Fetch and integrate remote changes.
git push -u origin <branch>	Publish a branch and set its upstream.
git restore <path>	Restore an unstaged working-tree file.

References

Git. (2026). Git reference. <https://git-scm.com/docs>

GitHub. (2026). GitHub Docs. <https://docs.github.com/>

GitHub. (2026). Open Source Guides. <https://opensource.guide/>

About the Author

Rinante M. Buntod is the author of this instructional module. Additional professional and academic biographical information may be added before publication.